DeepRx: Fully Convolutional Deep Learning Receiver

Mikko Honkala, Dani Korpi, and Janne M. J. Huttunen

***Abstract*— Deep learning has solved many problems that are out of reach of heuristic algorithms. It has also been successfully applied in wireless communications, even though the current radio systems are well-understood and optimal algorithms exist for many tasks. While some gains have been obtained by learning individual parts of a receiver, a better approach is to jointly learn the whole receiver. This, however, often results in a challenging nonlinear problem, for which the optimal solution is infeasible to implement. To this end, we propose a deep fully convolutional neural network, DeepRx, which executes the whole receiver pipeline from frequency domain signal stream to uncoded bits in a 5G-compliant fashion. We facilitate accurate channel estimation by constructing the input of the convolutional neural network in a very specific manner using both the data and pilot symbols. Also, DeepRx outputs soft bits that are compatible with the channel coding used in 5G systems. Using 3GPP-defined channel models, we demonstrate that DeepRx outperforms traditional methods. We also show that the high performance can likely be attributed to DeepRx learning to utilize the known constellation points of the unknown data symbols, together with the local symbol distribution, for improved detection accuracy.**

***Index Terms*— Radio receiver, deep learning, convolutional neural networks, 5G, channel estimation, equalization.**

I. Introduction

THE recent advances in deep learning techniques have resulted in new applications of neural networks in various fields, including wireless communications [1]–[6]. Machine learning (ML) has already been shown to be very effective in optimizing the higher layers of the communication stack [7]–[10]. However, it is indisputable that the foundation of the overall network-level performance is set by the processing employed in the physical layer. Therefore, in this article, we show that there are unrealized gains to be achieved also in the physical layer processing via the use of ML, with which we can improve the radio performance of the individual devices within the network.

In particular, by treating the radio receiver implementation as one supervised learning problem, it is possible to consider many of the individual receiver tasks, such as channel estimation and equalization, jointly. The thesis of this work is that this will result in a higher performance than optimizing each individual component separately, since the optimization target can closely mirror the real-world target, meaning that the resulting model is not bound by unrealistic or inaccurate assumptions. In this article, our approach is to train a deep neural network to detect the received bits from the received waveform. Namely, we consider the physical layer processing of a 5G-compliant radio receiver, whose task is to obtain the information bits from an orthogonal frequency-division multiplexing (OFDM) waveform, modulated in accordance with 5G numerology [11]. The benefit of this type of an approach is that the receiver's task can be represented as a supervised learning problem without requiring any labeling by existing algorithms or by humans. Indeed, the input data is simply the received waveform in the frequency domain, while the original transmitted bits are the corresponding labels.

There are already several studies that propose implementing certain parts of the digital receiver chain using a neural network. For instance, [6] considers a neural network-aided channel estimator, which is shown to approach the performance of a genie-aided estimator when the number of RX antennas is large, while the ML-based mmWave channel estimation solution proposed in [12] is shown to outperform existing compressed sensing-based schemes throughout the signal-to-noise ratio (SNR) range. In [13] Chang *et al.* apply convolutional neural networks (CNNs) [14]–[16] to equalization, achieving a lower error vector magnitude than that of multi-modulus algorithm or recursive least squares-based approaches. Deep learning-based demapping is analyzed in [17], where Shental and Hoydis propose a deep neural network approach for efficiently calculating bit log-likelihood ratios (LLRs) of equalized symbols. The proposed deep learning based demapper is shown to achieve similar accuracy as the optimal log maximum a-posteriori rule, albeit with greatly reduced computational cost. In addition, there are some works that propose augmenting traditional receiver processing flow with deep learning components to improve the performance [18]–[20], each of them outperforming the traditional receiver benchmark, as long as proper training is conducted.

The prospect of implementing larger portions of the receiver using a single neural network has also been considered by some authors. For instance, Ye *et al.* [21] investigate combined channel estimation and signal detection using deep learning. There, the detection is carried out using a fully-connected neural network that processes the pilots and the data signal. Such a fully learned receiver is shown to clearly outperform

Manuscript received May 5, 2020; revised October 2, 2020 and January 12, 2021; accepted January 12, 2021. Date of publication February 2, 2021; date of current version June 10, 2021. The associate editor coordinating the review of this article and approving it for publication was J. Gross. *(Corresponding author: Dani Korpi.)*

The authors are with Nokia Bell Labs, 02610 Espoo, Finland (e-mail: dani.korpi@nokia-bell-labs.com).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TWC.2021.3054520.

Digital Object Identifier 10.1109/TWC.2021.3054520

1536-1276 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on July 08,2026 at 03:37:59 UTC from IEEE Xplore. Restrictions apply.

a minimum mean square error (MMSE) based traditional receiver when there are few channel estimation pilots or when the cyclic prefix is omitted. In [5], on the other hand, CNNs are applied to implement a receiver that extracts the bit estimates directly from a time-domain RX signal, achieving excellent performance at low-to-mid SNRs. At high SNRs, the CNN-based scheme still outperforms a linear least squares-based receiver, while falling behind an MMSE-based receiver and a genie-aided receiver with perfect channel knowledge. As the most extreme case, deep learning-based end-to-end solutions, where both the transmitter and receiver are learned simultaneously from the data without any prespecified modulation scheme or waveform, have also been widely studied [3], [22]–[24]. Such schemes have been shown to have potential to outperform traditional heuristic radio links, e.g., by learning a better constellation shape [24].

Many of these prior works have successfully implemented a neural-network-based radio receiver and have also demonstrated high performance in comparison to the traditional receiver algorithms. These findings indicate that developing a data-driven receiver with deep neural network can indeed increase the performance of the future radio systems. However, we show that by carefully designing the neural network architecture and its inputs, it is possible to achieve an even higher increase in performance. Our findings indicate that most gains are obtained by allowing the neural network to utilize the unknown data symbols and their distribution in enhancing the channel estimation accuracy.

In contrast to many related works, we also consider standards compliance, in particular with 5G New Radio (NR). In order for the learned receivers to achieve 5G compliance, they must support, among other things, the various different demodulation reference signal (DMRS) or pilot configurations included in the NR specifications. In addition, the output of the receiver must be decoded by a low-density parity-check (LDPC) decoder, which means that the neural network must be capable of estimating also the uncertainty of each received bit for each modulation order. Consequently, there is a need for flexible deep learning-based receiver solutions that can handle all the different reference signal configurations and modulation schemes within a single implementation while being compliant with the other processing stages.

In this article, we propose a fully convolutional neural network architecture, referred to as DeepRx that learns a high-performance OFDM receiver from the data. This is achieved by feeding a frequency domain representation of the received signal over the whole transmission time interval (TTI) to a CNN, and training it to output the LLRs of the transmitted bits (soft bits) without restricting the individual processing stages in any way. Furthermore, the CNN-based DeepRx is trained to operate under various different scenarios, parameters and configurations. Therefore, with the proposed approach, it suffices to implement only the transmitter manually; the receiver can be learned based on the received waveform and the known transmitted bits. In this work, training and validation of the proposed architecture is carried out using simulated uplink (UL) data, generated with Matlab's 5G toolbox [25]. The performance of the different receivers is evaluated using the bit error rate (BER) both before and after LDPC decoding.

In particular, the main contributions of this article are as follows:

- We describe a novel deep learning receiver (DeepRx), which outperforms traditional receivers in terms of radio performance and is among the best reported in literature. The main reasons for the performance increase are: a) DeepRx is trained to obtain LLRs directly from frequency domain antenna data, which allows it to carry out all tasks (channel estimation, equalization and soft demapping) jointly, b) it is allowed to utilize, in addition to pilots, the received data symbols and their distributions during all the tasks, which is especially helpful under high Doppler shifts and sparse pilot configurations, c) the utilized training approach allows the receiver to cope well also with non-Gaussian noise, such as interference from another cell.
- We design the DeepRx network to be 5G-compliant in several aspects and show that it is possible for a single network to cover large parts of the 5G specification efficiently. The same network can operate under arbitrary DMRS configurations, modulation orders, and code rates.[1] We also demonstrate that the LLRs provided by the DeepRx network can be processed by a 5G-compliant LDPC decoder, resulting in state of the art performance.
- We provide extensive performance comparisons demonstrating that DeepRx can outperform the benchmark receiver algorithms. We also provide example results with carefully manipulated input signals to gain insight into its behavior and outstanding performance.

The rest of this paper is organized as follows. In Section II, we describe the reference transmitter and receiver architectures which are used as a basis for generating training and validation data. We also give a brief overview into traditional receiver architectures that are used as baselines in our study. Then, Section III discusses in detail the proposed DeepRx architecture and the training procedure. After this, Section IV outlines the simulation-based training and validation data generation, while Section V shows the validation results with comparisons against baseline receivers. Finally, Section VI concludes the article.

II. System Model

For data generation, we consider a 5G-compliant physical uplink shared channel (PUSCH) simulator, implemented with MATLAB's 5G Toolbox [25]. The simulator architecture is illustrated in Fig. 1 and it includes all the physical layer components, starting from the transport block (TB) information bits, and terminating after the decoding phase. The hybrid automatic repeat request (HARQ) procedure is omitted from the analysis of this article since the proposed DeepRx architecture is completely transparent to the processes of the

[1]It is to be noted that, even though particular attention is paid to 5G compliance, the proposed CNN architecture is compatible with any communication system employing OFDM waveforms, such as WiFi.

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on July 08,2026 at 03:37:59 UTC from IEEE Xplore. Restrictions apply.

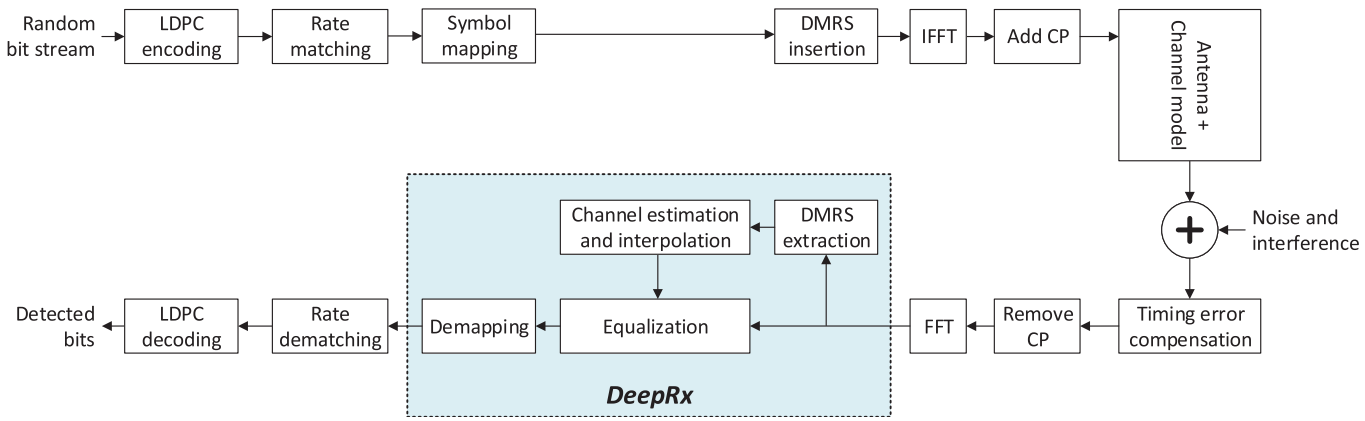


Fig. 1. Illustration of the link-level simulator used for data collection and benchmarking.

higher layers. Moreover, in this study, we restrict the number of transmit antennas to one, considering a single-input and multiple-output (SIMO) system. Extending the work to a multiple-input and multiple-output (MIMO) system is left as a future work item.

As a starting point, a specified amount of uniformly distributed information bits are randomly generated. These are then encoded with an LDPC code and fed through the rate matching processing. The resulting code word is mapped into symbols, and these symbols are distributed over the available physical resource blocks (PRBs) within the TTI. Demodulation reference signals (DMRS), or pilots, are also inserted into the specified subcarriers. After this, the data is turned into an OFDM waveform by feeding the PRBs into an inverse Fourier transform (IFFT), resulting in 14 individual OFDM symbols per each TTI. Before transmission, a cyclic prefix (CP) is added to the beginning of each OFDM symbol to mitigate inter-symbol-interference.

Having obtained the transmit waveform for the whole TTI, it is fed through a channel model. For the purposes of this work, we utilize ten different channel models specified by 3GPP [26]. They include five different clustered delay line (CDL) and tapped delay line (TDL) channel models, each with their own delay profile. Out of these ten models, four represent a non-line-of-sight (NLOS) scenario, while the remaining are line-of-sight (LOS) channel models. The channel model is chosen randomly for each frame. In addition, the maximum Doppler shift and root mean square (RMS) delay spread are also randomly chosen for each individual channel realization.

After propagating through the channel, the received waveform is subjected to observation noise (Gaussian white noise is assumed) and interference. In this article, the SNR is defined over the whole band, meaning that the SNR upon detection is somewhat higher since some of the subcarriers are unused.

Some of the studied experiments also include inter-cell-interference. The interference is represented as another waveform with similar numerology but different random information bits. Moreover, it is also fed through a different random channel realization and has a random time offset with respect to the desired signal. The power of the interference with respect to the desired signal (signal-to-interference ratio, SIR) is randomly chosen.

Having added the noise and interference, the total received waveform is fed to the receiver for detecting the information bits. The first stage in the receiver is timing offset estimation of the received waveform, which is done based on the known channel delay plus a random offset to model practical timing errors. After this, the signal is demodulated, which simply consists of removing the CP and calculating the fast Fourier transform (FFT) of each individual OFDM symbol. In this case, each TTI consists of 14 OFDM symbols, as already mentioned above. The received signal after the FFT can be expressed as

$$\mathbf{y}_{ij} = \mathbf{H}_{ij} x_{ij} + \mathbf{n}_{ij}, \tag{1}$$

where i and j denote the OFDM symbol and subcarrier indices, respectively, $\mathbf{y}_{ij} \in \mathbb{C}^{N_r \times 1}$ and $x_{ij} \in \mathbb{C}$ are the received and transmitted symbols, respectively, $\mathbf{H}_{ij} \in \mathbb{C}^{N_r \times 1}$ is the channel in the ith OFDM symbol over the jth subcarrier, $\mathbf{n}_{ij} \in \mathbb{C}^{N_r \times 1}$ is the noise-plus-interference signal, and N_r is the number of RX antennas. The total number of OFDM symbols is denoted by S (which in the context of 5G is fixed at $S = 14$, as mentioned above), while the total amount of subcarriers is denoted by F, meaning that $i = 0, \ldots, S-1$ and $j = 0, \ldots, F-1$.

The frequency-domain samples of the received 14 OFDM symbols, represented in (1), constitute the input of the actual receiver processing. In the following section, we explain how they are traditionally processed using least squares (LS) channel estimation and linear minimum mean square error (LMMSE) equalization (our baseline), while in Section III we describe how the processing can be carried out by a deep CNN (DeepRx).

Traditional Receiver Processing

The first step in OFDM receiver processing is to estimate the channel using the known pilots (DMRS) and assuming frequency-flat fading for the individual subcarriers. More precisely, using (1), the raw channel estimate is first calculated

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on July 08,2026 at 03:37:59 UTC from IEEE Xplore. Restrictions apply.

as (note that the pilots lie on the unit circle in the complex domain):

$$\widehat{\mathbf{H}}_{ij} = \mathbf{y}_{ij} x_{ij}^{*}, \quad (i,j) \in \mathcal{P} \tag{2}$$

where $\mathcal{P}$ denotes the set of indices corresponding to pilot locations in the time-frequency grid, and $(\cdot)^*$ denotes the complex conjugate. The raw channel estimate is then interpolated to fill the whole time-frequency grid and thereby provide channel estimates for the data symbols. This will result in the channel estimate $\widehat{\mathbf{H}}_{ij} \in \mathbb{C}^{N_r \times 1}$ for $(i,j) \in \mathcal{D}$, where $\mathcal{D}$ denotes the set of indices of the data symbols and subcarriers. Also the noise(-plus-interference) power σ_n^2 is estimated during the channel estimation phase.

Each data symbol is then equalized using the interpolated channel estimate. As mentioned above, an LMMSE equalizer is used in the considered reference receiver architecture, which means that the equalizer output for $(i,j) \in \mathcal{D}$ is given by

$$\hat{x}_{ij} = \left(\widehat{\mathbf{H}}_{ij}^{H} \widehat{\mathbf{H}}_{ij} + \hat{\sigma}_n^2 \mathbf{I} \right)^{-1} \widehat{\mathbf{H}}_{ij}^{H} \mathbf{y}_{ij}, \tag{3}$$

where $\hat{\sigma}_n^2$ is the noise power estimate, $\mathbf{I}$ is the identity matrix and $(\cdot)^H$ denotes the Hermitian transpose.

The equalized symbols are next fed to the demapper, which calculates the soft bits or LLRs based on the symbol estimates $\hat{x}_{ij}$. The LLRs are defined by

$$L_{ijl} \triangleq \log \left(\frac{\Pr\left(c_l = 0 | \hat{x}_{ij}\right)}{\Pr\left(c_l = 1 | \hat{x}_{ij}\right)} \right), \tag{4}$$

where $\Pr\left(c_l = b | \hat{x}_{ij}\right)$ is the conditional probability that the transmitted bit c_l is $b \in \{0,1\}$ given the observed symbol $\hat{x}_{ij}$, and $l = 0, \ldots, B-1$ where B is the number of bits per symbol. Assuming that the equalizer removes all the channel effects and only Gaussian white noise remains, the LLRs can be approximated with good accuracy by

$$L_{ijl} \approx \frac{1}{\hat{\sigma}_n^2} \left(\min_{x \in C_l^1} \|\hat{x}_{ij} - x\|_2^2 - \min_{x \in C_l^0} \|\hat{x}_{ij} - x\|_2^2 \right), \tag{5}$$

where $x \in C_l^b$ represent those points in constellation C for which the lth bit is $b \in \{0,1\}$, and $\hat{\sigma}_n^2$ is again the noise power estimate. In the reference implementation the final LLRs are also scaled by the channel magnitude of the considered subcarrier to reflect the higher uncertainty due to more severe fading.

III. Convolutional Neural Network-Based Receiver

Here we turn the focus to the design rationale of the DeepRx network architecture. DeepRx operates on the Fourier transformed frequency-domain data (see Fig. 1) collected during a TTI and its output are the final bit-level LLRs. Inputting the whole TTI at once allows the network to utilize all the information therein for estimating each of the bits. Moreover, given a non-static environment and potentially mobile UEs, the frequency-domain channel coefficients are different for each subcarrier and OFDM symbol. Considering that the physical channels in such cases are locally strongly correlated in frequency and in time, we employ a fully-convolutional neural network, where 2D convolutions operate in frequency and time dimensions. The objective of these 2D CNN filters is to learn such local correlations that are not frequency or time dependent, and re-use them effectively over the whole TTI.

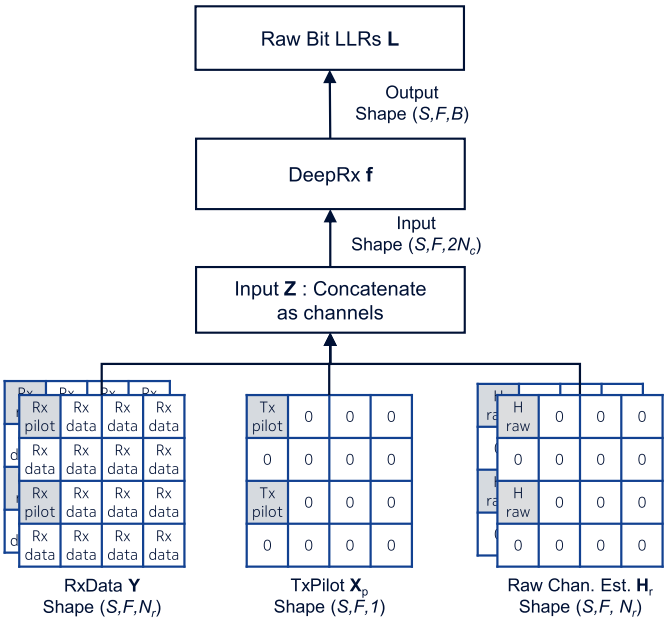


Fig. 2. Input to the DeepRx is a concatenation of the received unknown data, the known pilot symbols and the raw channel estimates.

Another design rationale is that, as the sparse pilot symbols only provide local information of the channel, we allow the network to utilize the unknown data and its known distribution for improved estimation of LLRs far away from the actual pilot locations. Therefore, we give the CNN unrestricted access to all data instead of designing separate blocks or paths for pilot-based channel estimation and data symbol equalization as in most related works. We assume that the channel and LLR estimation can be improved if the whole TTI (both the unknown received data and the known pilots) is inputted to the network in a coherent fashion, since this allows it to utilize all the data to carry out the assigned task.

While this design principle can also be extended to the case where several signal streams are spatially multiplexed in MIMO operation, the forthcoming description is written for a SIMO case where there is just one signal present. Extending the DeepRx architecture to full MIMO processing is an important future work item for us.

As shown in Fig. 2, DeepRx operates on a three-dimensional input array consisting of received data and pilot information, which is constructed as follows.

- The first part of input is the received signal after the FFT, denoted by $\mathbf{Y} \in \mathbb{C}^{S \times F \times N_r}$, which contains both data and received pilot symbols (recall that S is number of symbols in time, F is number of subcarriers, and N_r is the number of RX antennas).
- The second part is $\mathbf{X}_p \in \mathbb{C}^{S \times F}$ which contains the pilot reference symbols positioned so that they correspond to the pilot positions within the received signal $\mathbf{Y}$ in both frequency and time, the non-pilot positions being filled with zeros (see Fig. 2 for illustration).

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on July 08,2026 at 03:37:59 UTC from IEEE Xplore. Restrictions apply.

TABLE I
THE DEEPRX CNN RESNET ARCHITECTURE. THE RESNET BLOCK IS DESCRIBED IN FIG. 3

Layer	Type	Filter (S, F)	Dilation (S, F)	Output Shape
Input 1 $\mathbf{Y} \in \mathbb{C}$	RX Data			(S, F, N_r)
Input 2 $\mathbf{X}_p \in \mathbb{C}$	TX Pilot			(S, F, 1)
Input 3 $\mathbf{H}_r \in \mathbb{C}$	Raw channel estimate			(S, F, N_r)
Input $\mathbf{Z}_c \in \mathbb{C}$	Concatenate inputs 1-3 $\in \mathbb{C}$			(S, F, N_c)
Real input $\mathbf{Z} \in \mathbb{R}$	Concatenate $\in \mathbb{R}$			(S, F, $2N_c$)
Conv. In	2D convolution	(3,3)	(1,1)	(S, F, 64)
ResNet Block 1	Depthwise separable conv.	(3,3)	(1,1)	(S, F, 64)
ResNet Block 2	Depthwise separable conv.	(3,3)	(1,1)	(S, F, 64)
ResNet Block 3	Depthwise separable conv.	(3,3)	(2,3)	(S, F, 128)
ResNet Block 4	Depthwise separable conv.	(3,3)	(2,3)	(S, F, 128)
ResNet Block 5	Depthwise separable conv.	(3,3)	(2,3)	(S, F, 256)
ResNet Block 6	Depthwise separable conv.	(3,3)	(3,6)	(S, F, 256)
ResNet Block 7	Depthwise separable conv.	(3,3)	(2,3)	(S, F, 256)
ResNet Block 8	Depthwise separable conv.	(3,3)	(2,3)	(S, F, 128)
ResNet Block 9	Depthwise separable conv.	(3,3)	(2,3)	(S, F, 128)
ResNet Block 10	Depthwise separable conv.	(3,3)	(1,1)	(S, F, 64)
ResNet Block 11	Depthwise separable conv.	(3,3)	(1,1)	(S, F, 64)
Conv. Out	2D convolution	(3,3)	(1,1)	(S, F, B)
LLR Output $\mathbf{L}$	Output			(S, F, B)

- In addition, we pre-compute the raw channel estimate $\widehat{\mathbf{H}}_r = \mathbf{Y} \odot \mathbf{X}_p^*$ for the pilot positions, where $\odot$ and $(\cdot)^*$ are the element wise product and complex conjugate, respectively, and give this as the third part of the input. In the case where there are multiple RX antennas, the elements of $\mathbf{X}_p$ are duplicated along the third dimension when carrying out the raw channel estimation.

As the first two dimensions of $\mathbf{Y}$, $\mathbf{X}_p$ and $\widehat{\mathbf{H}}_r$ are equal, they can be stacked together along the third dimension (channel) to form $\mathbf{Z}_c \in \mathbb{C}^{S \times F \times N_c}$ where $N_c = 2N_r + 1$. Furthermore, we convert the complex-valued input into real-valued by stacking the real and imaginary parts of the input as separate channels, resulting in the final input array $\mathbf{Z} \in \mathbb{R}^{S \times F \times 2N_c}$. Stacking the relevant data into channels in this manner allows the convolutions to operate on data that is mutually related. While it is also possible to use a complex-valued network with a complex-valued input, we have not observed any performance gains in doing so.

Remark: The input $\mathbf{X}_p$ includes all required information about the pilot positions, which can have various different configurations. Via this, our setup facilitates a single network to operate with multiple pilot configurations, given that they have been presented to the network during training. We have experimentally verified this by training a single network that successfully handled all the pilot configurations defined in the 5G standard. It should also be noted that while it is not strictly necessary to feed the raw-channel estimate $\widehat{\mathbf{H}}_r$ as input to DeepRx, doing so allows for a somewhat easier and faster learning process (approximating such operation with a neural network can require several layers).

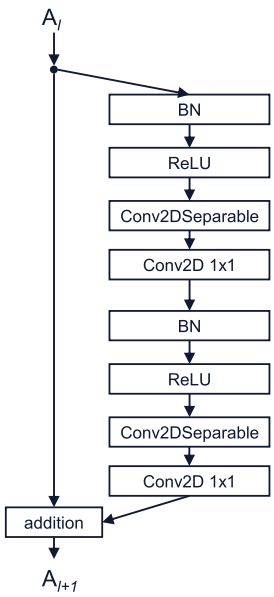
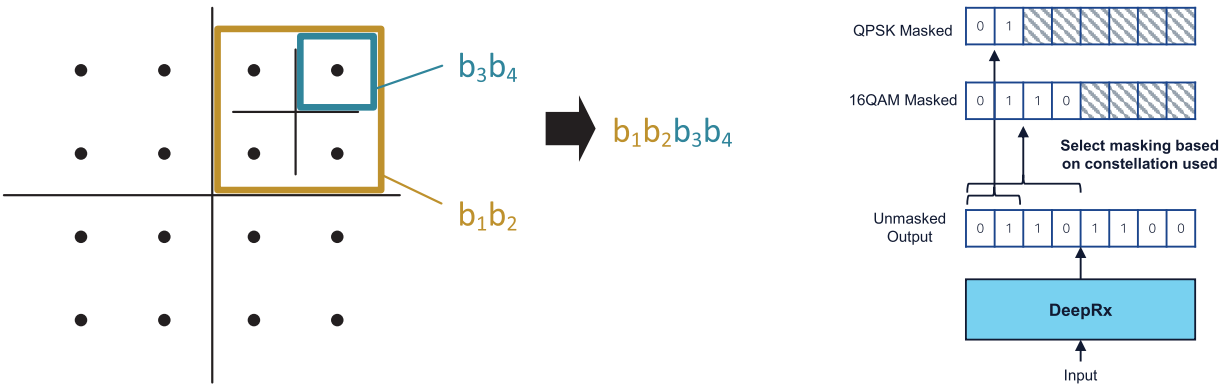


Fig. 3. Preactivation ResNet block between previous layer A_l and the next layer A_{l+1}. Blocks "Conv2DSeparable" and "Conv2D 1×1" form a depthwise separable convolution introduced in [28], [29]. BN and ReLU stand for batch normalization and rectified linear unit, respectively.

For the neural network $\mathbf{f} : \mathbf{Z} \rightarrow \mathbf{L}$, where $\mathbf{L}$ is the matrix of output LLRs with dimensions specified below, we employ a batch normalized CNN with residual connections using a preactivation ResNet [27], as described in Table I. The applied ResNet blocks are described in detail in Fig. 3. The 2D convolutions operate over the first two dimensions, time and frequency. The architecture is fully convolutional and therefore the size of its output ($S \times F$) is determined directly from the size of the input, and it can even vary for each TTI when using the same trained neural network. Namely, LLR estimates must be calculated for each symbol in the input and thus constant resolution ($S \times F$) is maintained throughout the network without resorting to max-pooling or striding, which are often used in CNNs to reduce resolution. Also note that the input size can be larger than what was used in the training, but not smaller than the total receptive field because the zero-padding for the convolutions might produce unpredictable effects in the output.

Instead of altering the resolution, we increase the amount of filters in the middle of the network and apply dilated convolutions to increase the receptive field as is often done, for example, in semantic segmentation [30]. Using dilation instead of striding enables the network to retain the detailed information about each input symbol while still allowing it to capture longer dependencies in time and frequency.

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on July 08,2026 at 03:37:59 UTC from IEEE Xplore. Restrictions apply.



(a) The relationship between 16-QAM and QPSK constellation points.

(b) Masking bits based on modulation.

Fig. 4. (a) The hierarchical relationship between QPSK and 16-QAM: In QPSK, the two bits b_1 and b_2, are defined by the complex quadrant of the symbol. In 16-QAM, this also holds for the first two bits (orange box), while the last two bits, b_3 and b_4, correspond to the finer location within the quadrant (blue box). (b) Bit masking for supporting multiple QAM constellations in DeepRx.

The dilations proved to be especially important for the more shallow architectures. We also observed improved results when using depthwise separable convolutions [28], [29] instead of normal convolutions. The main results in this paper are computed with a depth multiplier value of 2 for the depthwise separable convolutions, which simply means that the number of output channels in the depthwise convolution is doubled in order to increase the number of parameters and thereby improve the modeling capability of the network. However, it was observed that using the value of 1 works also well. Section V-C contains a detailed ablation study related to these and various other architectural choices.

Finally, the prediction of bits is simply modeled as a binary classification problem. The final output of the DeepRx consists of the bit LLRs $\mathbf{L} \in \mathbb{R}^{S \times F \times B}$, where B is the number of bits in the used constellation (e.g., 4 for 16-QAM). For training, we use the binary sigmoid cross-entropy (CE) loss between each of the ground truth bits and the output $\mathbf{L}$ of the network,

$$\mathrm{CE}(\boldsymbol{\theta}) \triangleq -\frac{1}{\#\mathcal{D}B} \sum_{(i,j)\in\mathcal{D}} \sum_{l=0}^{B-1} \Big(b_{ijl} \log(\hat{b}_{ijl}) + (1 - b_{ijl}) \log(1 - \hat{b}_{ijl}) \Big) \quad (6)$$

where $\#\mathcal{D}$ is the number of resource elements carrying data and $\hat{b}_{ijl}$ is an estimate for the probability that the bit b_{ijl} is one,

$$\hat{b}_{ijl} = \mathrm{sigmoid}\left(L_{ijl}\right) = \frac{1}{1 + e^{-L_{ijl}}}. \quad (7)$$

Remark: Even though the actual bits are used as the ground truth of outputs in the cross-entropy loss, we consider the LLRs ($\mathbf{L}$) as the output of the inference network. The LLRs represent also the model's uncertainty about the bits and can be fed, for example, to an LDPC decoder, which then makes the decisions regarding the actual information bits.

The output of DeepRx has been designed so that a single network can be trained to support multiple quadrature amplitude modulation (QAM) schemes. As shown in Fig. 4a, the QAM schemes in 5G (QPSK, 16-QAM, 64-QAM, 256-QAM) are related to each other hierarchically so that it is possible to map four constellation points in the higher-order modulation to a single constellation point in the lower-order modulation. We take advantage of this relation and define the output of the model in such a way that same output bits/LLRs correspond to the same part of the constellation space, regardless of the modulation used. This means that, for an individual symbol, the first two outputted bits describe the complex quadrant of the symbol (1st-order point), the next two bits define the quadrant around the 1st-order point (giving the 2nd-order point), etc. In practice, this is achieved by setting the number of outputs according to the highest supported modulation order, in this case choosing 8 outputs to support up to 256-QAM modulation. During training and inference, the outputs are then masked, as depicted in Fig. 4b, such that only the bit positions actually used in the modulation scheme are considered. During training, the masked-out bits do not affect the computed loss for that symbol, while during inference the masked-out bits are simply omitted. With this approach, a single network can learn to support all modulation orders.

IV. Generation of Training and Validation Data

The training and validation data is generated with the link-level simulator implemented with Matlab's 5G Toolbox, which is modeling a 5G PUSCH link as described in Section II. The parameter values used in the simulations are listed in Table II. Each individual data set contains 500 000 TTIs, of which 60% is used for training, and a subset of the remaining 40% is used for validation. As already mentioned, the HARQ procedure is purposefully omitted from the simulations since it has implications only for the processing occurring after calculating the LLRs. This renders the proposed DeepRx architecture transparent to the HARQ process.

The randomization of the parameters is done for each frame (period of 10 TTIs with the utilized numerology), using the ranges and distributions indicated in Table II. Also the SNR and SIR are randomized, although it should be noted that interference is present only in some of the results. Moreover, we wish to emphasize that drawing the logarithmic SNR from

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on July 08,2026 at 03:37:59 UTC from IEEE Xplore. Restrictions apply.

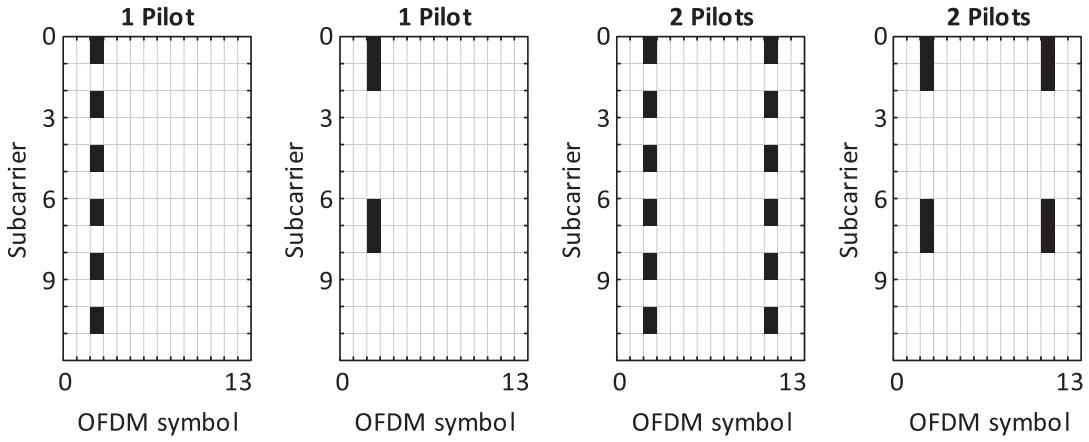


Fig. 5. The considered DMRS/pilot configurations, illustrated for one PRB over the duration of a TTI. Note that in the forthcoming results the pilot configurations are only differentiated in terms of how many OFDM symbols they utilize.

TABLE II
SIMULATION PARAMETERS FOR TRAINING AND VALIDATION

Parameter	Training	Validation	Randomization
Carrier frequency	4 GHz		None
Channel model	CDL-B, CDL-C, CDL-D, TDL-B, TDL-C, TDL-D	CDL-A, CDL-E, TDL-A, TDL-E	Uniform
RMS delay spread	10 ns – 300 ns		Uniform
Max. Doppler shift	0 Hz – 500 Hz		Uniform
SNR	−4 dB – 32 dB		Uniform
SIR	0 dB – 36 dB		Uniform
Number of PRBs	26 (312 subcarriers)		None
Subcarrier spacing	15 kHz		None
OFDM symbol duration	71 μs		None
TTI length	14 OFDM symbols / 1 ms		None
Modulation scheme	16-QAM		None
Code rate	$\frac{658}{1024}$		None
No. of TX antennas	1		None
No. of RX antennas	2		None
DMRS configuration	Four options, see Fig. 5		Uniform

a uniform distribution is a conscious choice to make the training more efficient for the full considered SNR range. It is well known that in reality SNR is more likely to follow a log-normal distribution.

As for the DMRS configuration, four different options are included in the data, each of them illustrated in Fig. 5 for a single PRB. For each frame of 10 TTIs, one of these configurations is chosen randomly. Note that in the validation results the four pilot configurations are only differentiated in terms of how many OFDM symbols include pilots (i.e., either one or two). We refer to these cases as "one pilot" or "two pilots" in the following. The reason for this is that we have not observed the frequency domain positioning of DMRS symbols to have any significant impact on the performance, and differentiating between them in the figures would only clutter the results without bringing any new insights. It should also be noted that only the DMRS configurations with two pilots are feasible for traditional LMMSE receivers, given the rather large Doppler range. The configurations with one pilot are included in the study only in order to evaluate the performance boundaries of DeepRx.

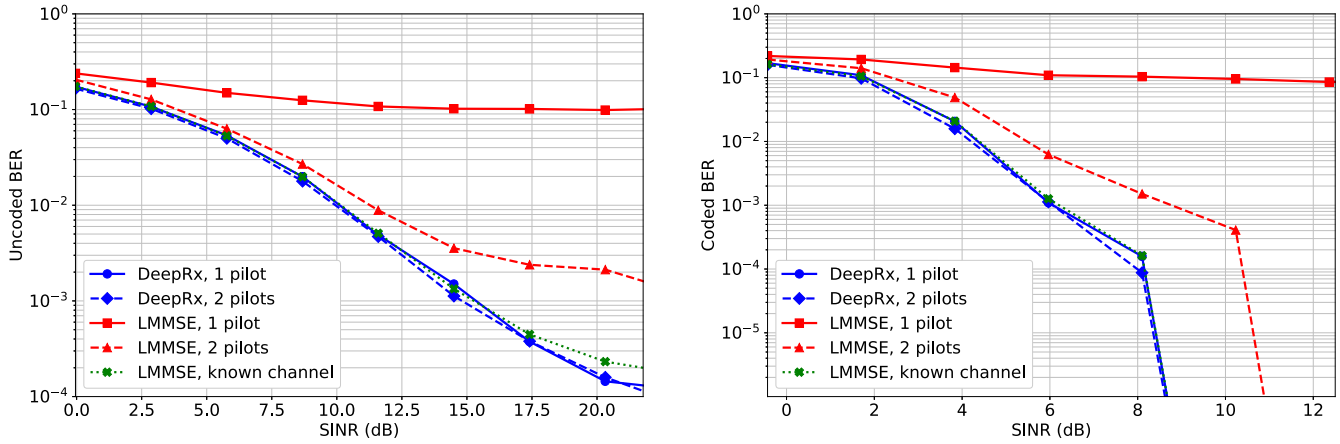
The simulated data sets are generated using ten different 3GPP channel models, whose detailed descriptions can be found in [26]. The models ending with letters *A*, *B*, and *C* represent NLOS, while those ending with *D* and *E* are constructed for LOS. It is also important to note that the models *CDL-A*, *CDL-E*, *TDL-A*, and, *TDL-E* are used only for validation, while the training is performed with the remaining six channel models listed in Table II (hence the 60%/40% split between training and validation data). This ensures that the deep neural network cannot achieve its high performance by simply learning the characteristics of the individual channel models.

In addition, we also generate a smaller training data set using a fully synthetic channel model, where the channel for an individual OFDM symbol is a 7-tap Rayleigh fading channel. After each OFDM symbol, the channel is randomly changed such that 90% of the variance of the new channel consists of the previous channel realization, while 10% of the variance is stemming from a new randomly generated 7-tap channel. This ensures a certain level of channel correlation between consecutive OFDM symbols. The purpose of this artificial channel model is not to represent a realistic propagation channel, but simply capture some fundamental properties of a wireless channel while still being different from the realistic 3GPP channel models. This allows for some insightful experiments as reported in more detail below in Section V-B. However, it should be emphasized that in the forthcoming results the 3GPP channels are used for both training and validation unless mentioned otherwise.

V. RESULTS

In this section, we compare the proposed CNN-based receiver, DeepRx, to two conventional LMMSE (Section II) receivers:

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on July 08,2026 at 03:37:59 UTC from IEEE Xplore. Restrictions apply.



(a) Uncoded BER (b) Coded BER

Fig. 6. (a) Uncoded BER and (b) coded BER performance of the DeepRx compared to the reference LMMSE receivers, without inter-cell-interference.

- One that performs LS channel estimation and interpolates the channel estimate over the data symbols and subcarriers;
- One that obtains the full channel information as a priori knowledge.

The former represents a realistic benchmark and is therefore referred to as a practical LMMSE receiver, while the latter one estimates the upper bound of the achievable performance with LMMSE equalization. Since we observed that approximating the LLRs with (5) has only a negligible impact on the final receiver performance, the approximated demapping rule is used in both the benchmark receivers, including the one with full channel knowledge. It should also be emphasized that the LMMSE receiver with full channel knowledge is resorting to otherwise practical OFDM receiver processing, meaning that it is not always able to achieve perfect equalization. This is primarily due to inter-carrier-interference caused by Doppler spread and the underlying channel changing also within each resource element.

We use bit error rates (BER) as the main performance criteria throughout this section. In particular, we consider two types of BERs: 1) a "raw" BER based on the hard decision (bit is 1 if LLR < 0, or 0 otherwise) referred to as the uncoded BER, and 2) the coded BER obtained by feeding the LLRs through a 5G-compliant LDPC decoder (preceded by a rate dematcher) and comparing the decoded bits to the original bit sequence. Investigating the coded BER reveals whether the LLRs provided by the DeepRx sufficiently capture the uncertainty of the detected bits for the purposes of LDPC decoding. The coded BER can also be seen as a proxy to the accuracy of the LLRs themselves, as direct evaluation of LLRs is difficult due to lack of ground truth (there is no explicit formula for the ideal LLRs under the utilized channel models).

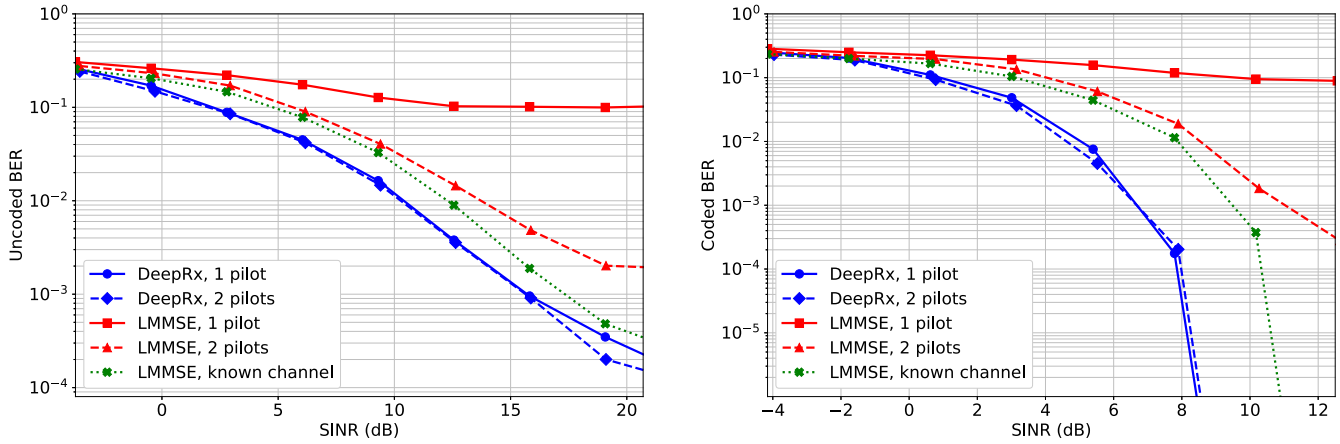
The model is trained using the simulated link-level data described in Section IV. The optimization is carried out using the LAMB optimizer [31], starting from a random initialization with the main learning rate set to 10^{-2}. We also apply a small weight decay with the scaling factor of 10^{-4} that prevents the weight magnitudes from growing during a long training run. LAMB allows for scaling the training to larger batch sizes (e.g., 80 TTIs, each TTI having $312 \times 14 \times 8$ bits, altogether $\sim$2,8M bits per batch, using four 2080Ti GPUs in parallel), while for smaller batch sizes (e.g., 20 TTIs), AdamW [32] might also suffice. For large batch sizes, we also use a linear learning rate warmup period from zero to main learning rate, with the duration of 800 iterations. In addition, the learning rate is decayed linearly to zero after reaching 30% of the total iterations. In total, we usually run 30k iterations with a batch size of 80 TTIs, and we have not observed significant improvement with longer runs. Overfitting was not observed with our models using the datasets described above, meaning that DeepRx was not observed, for instance, to memorize the training samples and thereby compromise its performance under validation.

A. Primary Validation Results

First, we consider performance of DeepRx without inter-cell-interference. From Fig. 6a, which shows the uncoded BER, it can be seen that DeepRx clearly outperforms the practical LMMSE receiver performing LS channel estimation. In fact, the CNN-based DeepRx can essentially match the performance of the LMMSE receiver with *full channel knowledge*, even when it has just one pilot symbol in time at its disposal. Due to the rather wide Doppler shift range within the data, the practical LMMSE receiver performs very poorly with just one pilot since it requires two pilots to operate reliably under such conditions. This is in stark contrast to DeepRx, for which, at higher SINRs, just one pilot is enough to outperform by a factor of 10 the practical LMMSE receiver having two pilots at its disposal. The reasons behind such exceptionally high performance are investigated further in Section V-B.

In addition, the coded BERs (Fig. 6b) indicate that the LLRs calculated by DeepRx are of sufficient quality for the LDPC decoder as the coded BER matches that of the LMMSE receiver with full channel knowledge. The gain over the practical LMMSE receiver with two pilots is roughly 2 dB,

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on July 08,2026 at 03:37:59 UTC from IEEE Xplore. Restrictions apply.



(a) Uncoded BER (b) Coded BER

Fig. 7. (a) Uncoded BER and (b) coded BER performance of the DeepRx compared to the reference LMMSE receivers, with inter-cell-interference.

whereas the practical LMMSE receiver does not even reach the waterfall region of the code with one pilot. The performance of DeepRx suffers only a marginal reduction when the number of pilot symbols is reduced from two to one.

Next, we consider a case with some inter-cell-interference in the received uplink signals. The interfering signal has a similar waveform as the signal of interest, but it has a random time offset and a different channel realization. The power of the interference is on average 4 dB lower than the noise power. The results are shown in Fig. 7. It can be observed that all the reference LMMSE receivers suffer from reduced performance as they do not have any capabilities for minimizing the effects of the interference. On the other hand, since DeepRx is trained with data that includes similar interference levels, it implicitly learns to manage the interference and outperforms even the LMMSE receiver with full channel knowledge. However, we wish to emphasize that this interference-related performance gap could likely be reduced if the LMMSE receivers were utilizing, for example, some type of interference rejection combining (IRC) [33].

Under inter-cell-interference, the performance gain of DeepRx compared to the reference receivers is even higher when coded BER is considered (Fig. 7b). This likely stems from the fact that the traditional demapper assumes Gaussian-distributed white noise in the calculation of LLRs, while the rather strong interference signal invalidates this assumption. DeepRx does not resort to such assumptions as it implicitly learns the proper receiver procedure for the given noise-plus-interference distributions, based on the training data it observes. Consequently, it outperforms even the benchmark with full channel knowledge by 2 dB, while the practical LMMSE receiver fails to even enter the waterfall region of the code within the considered SINR range.

B. Exploring the Reasons Behind DeepRx's Performance

The exceptionally high performance of DeepRx obviously raises questions as to what the CNN actually learns to do. In this section we will discuss some of our experimental findings that provide some insight into this.

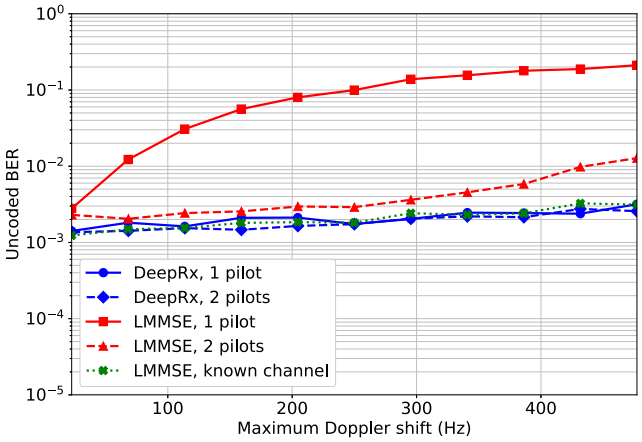
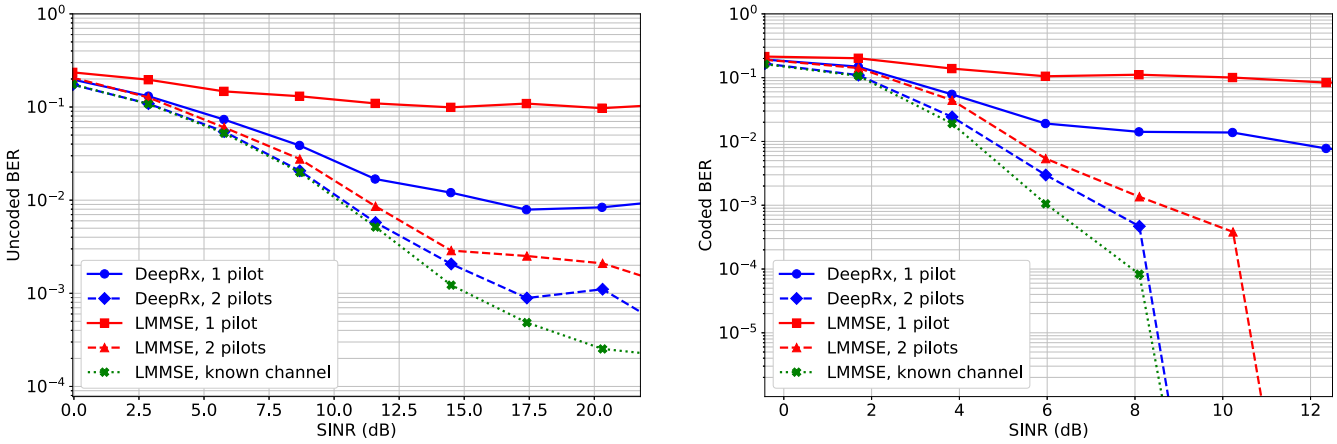


Fig. 8. Uncoded BER with respect to the maximum Doppler shift of the channel, calculated for SINRs sampled uniformly between 10–20 dB and without inter-cell-interference.

1) Temporal Tracking of the Channel: Figs. 6 and 7 showed very high performance with just one pilot symbol, positioned in the first half of the TTI. This suggests that the CNN-based DeepRx is capable of exceptionally accurate temporal tracking of the channel. To investigate this aspect in more detail, Fig. 8 shows the uncoded BER with respect to the maximum Doppler shift of the channel, which is inversely proportional to the coherence time of the channel. As can be expected, the practical LMMSE receiver with one pilot suffers from a rapid degradation of BER when the Doppler shift increases, as it must assume that the channel remains constant throughout the TTI. With two pilots, even the practical LMMSE receiver is capable of tracking the channel rather well up to Doppler shifts of 400 Hz, although DeepRx outperforms it throughout the considered Doppler shift range. In fact, even with the highest considered Doppler shift of 500 Hz, DeepRx using just one pilot symbol can match the BER of the LMMSE receiver with full channel knowledge. This corresponds to a UE velocity of roughly 135 km/h, representing already a case of rather severe

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on July 08,2026 at 03:37:59 UTC from IEEE Xplore. Restrictions apply.



(a) Uncoded BER (b) Coded BER

Fig. 9. (a) Uncoded BER and (b) coded BER performance of the DeepRx trained with a synthetic channel model and validated using the 3GPP channel models, compared to the reference LMMSE receivers, without inter-cell-interference.

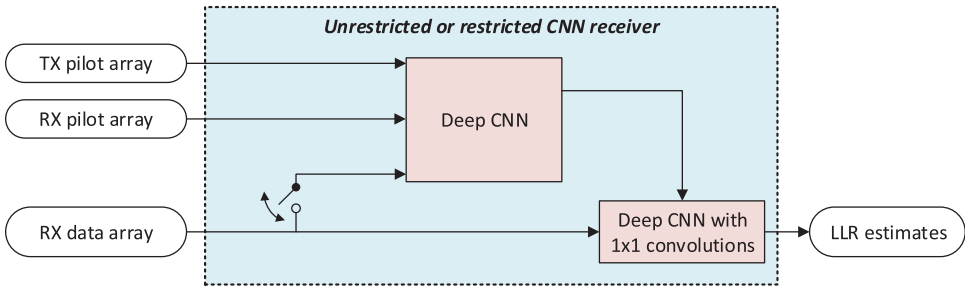


Fig. 10. Two alternative CNN architectures for investigating the behavior of DeepRx.

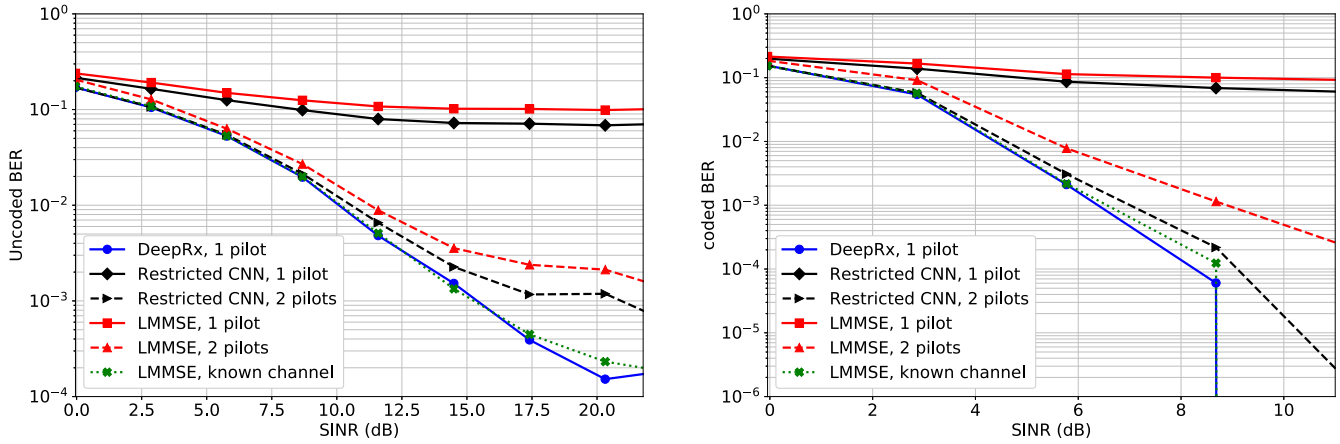
mobility. Note that the slight increase of BER for DeepRx and LMMSE receiver with full channel knowledge under the higher Doppler shifts is likely due to the frequency-domain spreading of the subcarriers, which reduces their orthogonality.

2) DeepRx Does Not Cheat by Learning Channel Models: Even though the channel models used in training are different from those used for validation, which prevents DeepRx from taking unfair advantage of the properties of individual channel models, it is still a possibility that the channel models share some modeling deficiencies which it can exploit. In order to ensure that no such exploitation occurs, we also trained DeepRx using the synthetic Rayleigh channel model described in Section IV, which is completely independent from the 3GPP channel models. Figure 9 shows the performance of such a model validated with the 3GPP channel models listed in Table II. Although the performance of the newly trained DeepRx is somewhat worse than when trained with the realistic 3GPP channel models, it still clearly outperforms the practical LMMSE receiver. For instance, with one pilot symbol, its BER performance is still an order of magnitude better than that of the practical LMMSE receiver. These results indicate that DeepRx indeed learns a generic solution for bit detection as it can be successfully applied to such different channel models. It is also likely that the performance deficit of the model trained with synthetic data is simply a result of the artificial behavior of the synthetic channel model, which prevents DeepRx from learning a fully accurate interpolation rule for the channel estimate.[2]

3) Blind Utilization of the Unknown Data During the Detection Process: To gain more insight into the reasons behind DeepRx's performance, let us train a more restricted CNN architecture. Fig. 10 shows two alternatives where the RX data is routed differently through CNN. The two alternatives differ in whether or not the data-carrying RX resource elements are fed to a deep CNN whose receptive field covers several subcarriers and symbols in time. This is depicted with a simple switch in Fig. 10, where a closed switch indicates that the deep CNN has full view of also the RX data array, in addition to the pilots. If the switch is open, on the other hand, the deep CNN has only access to the received and transmitted pilots, based on which it can perform the channel estimation. The equalization and demapping is carried out by a head of 3 layers of 1×1 convolutions (32 channels each), whose weights are also learned from the data. Such 1×1 convolutions facilitate equalization and demapping only in a symbol-by-symbol manner, i.e., the restricted CNN cannot

[2]We wish to emphasize that we cannot claim that a neural network would carry out channel estimation or equalization in the traditional sense. Determining exactly what happens inside a trained neural network is an open question. When referring to DeepRx, we simply use these terms to reflect on the internal processing of pilots and RX data which is necessary for performing bit detection.

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on July 08,2026 at 03:37:59 UTC from IEEE Xplore. Restrictions apply.



(a) Uncoded BER (b) Coded BER

Fig. 11. (a) Uncoded BER and (b) coded BER performance of the restricted CNN Receiver which can only process one data symbol at a time after pilot-based channel estimation, compared to the unrestricted DeepRx and reference LMMSE receivers, without inter-cell-interference.

utilize any spatial or temporal correlations in the RX data to improve the bit estimates. Furthermore, we wish to point out that the CNN architecture of Fig. 10 with the switch closed is essentially identical in performance and architecture to the primary DeepRx architecture presented in Section III, except for the additional 1×1 convolutional layers, which were added to ensure that it can be directly compared with the restricted CNN architecture. Therefore, for clarity, we shall refer to the unrestricted case as DeepRx in the forthcoming discussion. Moreover, we wish to point out that the restricted CNN architecture of Fig. 10 is somewhat representative of the existing deep learning-based channel estimators [6], [12], and therefore the corresponding performance comparisons will provide also some insight into the benefits of DeepRx when compared against these prior deep learning-aided receivers.

The BER results corresponding to the two architectures of Fig. 10 are shown in Fig. 11. For simplicity, the BER of DeepRx is only shown for the case of one pilot to ensure the readability of the figure. Altogether, it can be observed that the performance gain of the restricted CNN receiver over the practical LMMSE receiver is rather marginal. With one pilot, the BER of the restricted CNN receiver remains very high, whereas the unrestricted DeepRx is on par with the LMMSE having full channel knowledge, as observed already earlier. When there are two pilot symbols, the restricted CNN receiver fares better but is still clearly outperformed by DeepRx. These observations indicate that a crucial aspect for the high performance of DeepRx is to ensure that the CNN has full access to the data resource elements. In other words, one should not impose too many restrictions on how the deep CNN processes the RX signal to obtain the LLR estimates, such as limiting the deep learning processing to only channel estimation. Namely, it is to be expected that considering all the tasks jointly results in higher performance than learning and performing them separately. This way DeepRx can freely learn an extremely accurate reception procedure which might differ from the processing flow of a traditional receiver. Finally, we

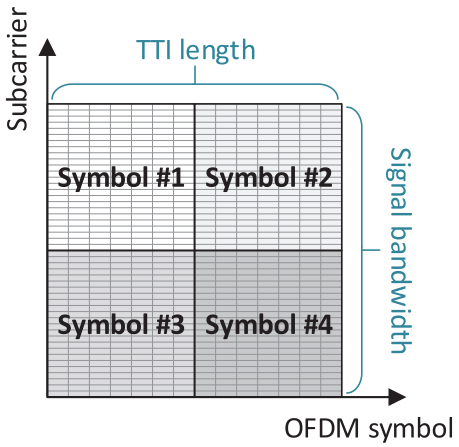
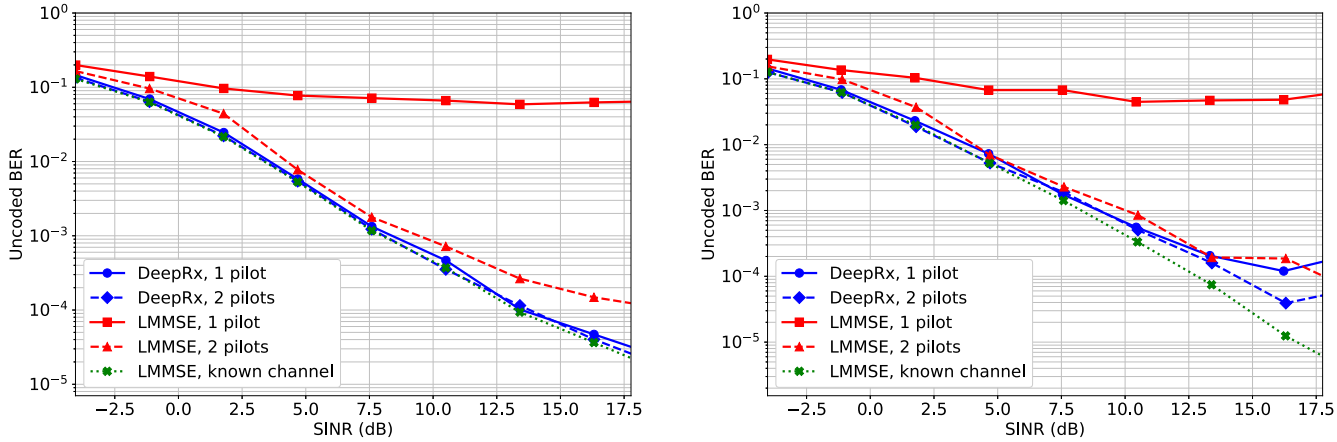


Fig. 12. Artificial transmit symbol distribution used as validation data for probing the behavior of DeepRx.

note that we experimented also with larger and deeper 1×1 convolutional heads, and the results were similar, so the aforementioned performance differences cannot be attributed to a smaller network handling the data symbols.

4) Advanced Utilization of Data Symbol Distribution: Another reason behind the high performance of DeepRx is that it might learn to utilize distributional information of data symbols (e.g., the known constellation points) to track the changes in the channel through time and frequency. For instance, DeepRx could learn to utilize the local distribution of symbols for blindly scaling and rotating the received data symbols to better match the known properties of the used constellation (compression and encryption of the transmitted data implicitly enforce a nearly uniform symbol distribution). To study this hypothesis, let us consider QPSK-modulated data in which each quadrant of the time-frequency grid of a TTI is filled with an identical symbol as shown in Fig. 12, while still retaining the regular pilot symbol positions. We validate

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on July 08,2026 at 03:37:59 UTC from IEEE Xplore. Restrictions apply.



(a) Regular QPSK data

(b) QPSK data with manipulated transmit symbol distribution

Fig. 13. (a) Uncoded BER after training and validating the DeepRx with regular QPSK data, and (b) uncoded BER after validating the DeepRx with QPSK data whose transmit symbol distribution is as depicted in Fig. 12.

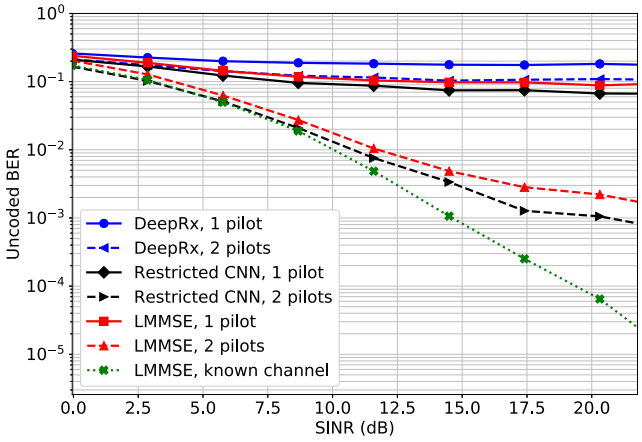


Fig. 14. Uncoded BER of DeepRx and the restricted CNN receiver with 16-QAM data where transmit symbols are allocated as depicted in Fig. 12. The models are the same that have been validated also in Figs. 6 and 11.

DeepRx[3] with this type of data and compare the performance to a case where the validation data has a typical (nearly uniformly random) distribution of QPSK symbols. As shown in Fig. 13, there is only a small performance reduction with this artificial validation data: DeepRx still clearly outperforms the practical LMMSE receiver, especially in the single-pilot case.

Next, we repeat the same experiment with 16-QAM constellation. Figure 14 shows results for a validation in which the data is constructed such that the time-frequency quadrants of each TTI are allocated with four randomly chosen 16-QAM symbols as shown in Fig. 12. Figure 14 also includes results for the restricted CNN receiver described above. Now, when both magnitude and phase of a symbol are used to encode information (as opposed to QPSK where all the information is in the phase of the symbol), the results are very different. Namely, DeepRx fails to deliver sufficient performance under the manipulated transmit symbol distribution. Moreover, the restricted CNN receiver, which is not able to learn to utilize the data symbol distribution since it cannot observe multiple symbols at once, clearly outperforms the (unrestricted) DeepRx architecture.

These findings indicate that DeepRx learns to rely on the data symbol distribution to perform some type of local magnitude normalization or magnitude tracking to accurately equalize the channel amplitude response, and is now fooled when the distribution is artificially violated in the validation data. Indeed, the restricted CNN architecture, which can only process one symbol at a time, is not affected by this. This failure seems to be mostly related to the amplitude of the symbols, indicating that DeepRx might have learned some type of a blind equalization scheme that bears resemblance to the well-known constant-modulus algorithm (CMA) [34]. This becomes particularly evident when considering the accuracy of the individual bits detected by DeepRx, averaged over the whole SINR range. It is observed that the two most significant bits (corresponding to the phase of the symbol; see Fig. 4a) remain very accurate (BER: 9.0×10^{-3}), while the two least significant bits (requiring also magnitude information of the symbol) are very inaccurate under a manipulated data symbol distribution (BER: 3.4×10^{-1}).

5) Comparison to Iterative Receiver Processing: Finally, we investigate further a hypothesis that DeepRx learns a reception technique which utilizes information about the legal constellation points. Such information is employed, for instance, in iterative receiver processing (see, e.g., [35]). In order to gain further insight into this hypothesis, we created an additional data set where the channel is just a single uniformly distributed random phase shift for the whole TTI. This means that the channel for each TTI is essentially a scalar on the open interval of $(0, 2\pi)$. To make the task of channel estimation still sufficiently demanding, we use only a single pilot symbol

[3]This version of DeepRx is trained with regular data with uniform random symbols. We did not include this particular type of artificial data (Fig. 12) to the training data as it would lead to a task too easy to learn due to the known repetition of symbols.

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on July 08,2026 at 03:37:59 UTC from IEEE Xplore. Restrictions apply.

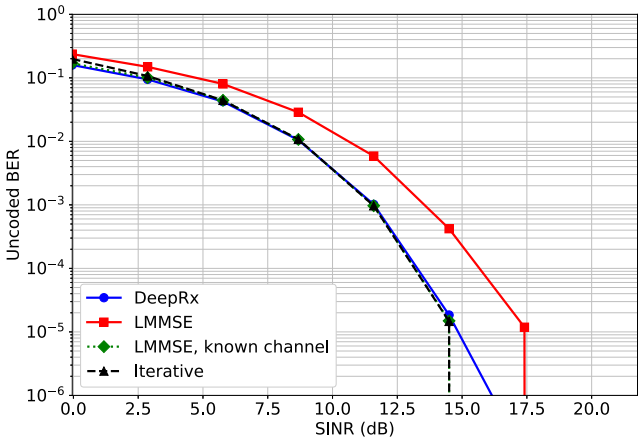


Fig. 15. Uncoded BER after training and validating the DeepRx with data where the channel is just a random phase rotation.

located at the center subcarrier of the 3rd OFDM symbol in time.[4] For this experiment, we also implemented a simple iterative receiver, which is using its initial symbol decisions as additional information to further refine the channel estimate. More precisely, it performs the initial channel estimation using the single pilot symbol and equalizes the RX symbols, after which it calculates a new channel estimate using also the previous RX symbol decisions. This refined channel estimate is then averaged over the whole TTI, after which it can be used for equalizing the RX symbols again. This procedure is repeated 40 times to ensure that convergence is achieved.

DeepRx is executed with an otherwise identical configuration and architecture as earlier, except for increasing the sizes of the convolutional filters and lengths of dilations: all (3,3)-convolutional filters are changed to (10,3)-filters and dilations in the frequency direction are increased such that the maximum dilation 6 is increased to 16 and other dilations are increased with a similar ratio (cf. Table I). This is done to ensure the visibility of the pilots as each TTI now contains just one pilot subcarrier, unlike in the primary scenario where the pilots span multiple subcarriers. Furthermore, we input two additional 2D arrays as channels to the network, one filled with the used frequencies and the other filled with the used time slots (both normalized to the unit interval). This allows the network to utilize time and frequency information, allowing the convolutional filters to specialize to this specific case of a single pilot.[5]

The resulting uncoded BER performance for the different receivers is shown in Fig. 15. Both the DeepRx and the iterative receiver are able to detect the channel almost perfectly, since they can utilize the unknown data symbols for channel estimation. On the other hand, the practical LMMSE receiver can only utilize a single pilot symbol for a noisy channel estimate, and thus falls short of the other solutions. This indicates that the processing learned by the DeepRx most likely resembles that of iterative receivers.

C. Ablation Studies and Notes on Complexity

Finally, to better understand the relationship between the exact architecture of DeepRx and its performance, we experimented with different CNN architectures and their hyperparameters. The results of this study are documented in Table III. The networks were trained with the full SINR range, while the BER values of the table are averages calculated for the SINR range from 15 dB to 20 dB for more informative results. The study was first performed using 16-QAM data with interference, after which we repeated it using the same hyperparameters and data without interference. Therefore the no-interference results can be considered as a separate test set. We also verified the lack of overfitting to the validation sets with a second test set generated with different random seed and confirmed that there was no significant difference between the results.

For most of the runs, we have used the same optimizer hyperparameters defined earlier in Section V. The notable exception is the model which uses normal convolutions instead of depthwise separable convolutions, where we had to divide the main learning rate by 2 in order for the model to converge.

Overall, it is clear from Table III that, with interference, almost all of the tested architectures outperform the LMMSE baselines. When tested with data without interference, the margin is smaller, but for cases with only one pilot, the performance gains from the CNN receivers are still considerable. For the remainder of this section, we consider the uncoded BER with one pilot and interference the primary benchmark metric for the different CNN architectures.

Regarding number of ResNet blocks (i.e., different depths), it seems that 11 blocks provide good performance under all scenarios, while still allowing for shrinking the network in terms of layer widths for more efficient inference. With dilations, it is possible to get relatively good performance with as few as 3 or 5 ResNet blocks, but the performance degrades slightly with one pilot and interference (1.63×10^{-3} for 3 blocks vs. 6.23×10^{-4} for 11 blocks). In general, the dilation experiments indicate that the network requires a certain receptive field size to function well, although the deeper 11-block networks work well also without dilations (6.91×10^{-4} without dilations vs. 6.23×10^{-4} with dilations).

Let us then investigate the effect of number of parameters, which is roughly proportional to the computational cost of inference for a given neural network (within certain conditions on the network architecture, such as number of layers and type of convolutions). Considering a CNN architecture of same depth of 11 ResNet blocks but with different widths, it is evident that roughly 1.2M parameters (primary DeepRx model with 64-256 convolutional channels, i.e., convolutional filters) provides a good balance between network complexity and BER performance. If only 0.1M parameters are used (model 11 S3 with 16-64 convolutional channels), the performance is heavily degraded. We also tried bigger networks, for example,

[4]This is not to be confused with the one pilot cases presented in Fig. 5 and used in the other results, where the pilot covers one OFDM symbol but multiple subcarriers. In this particular experiment, each TTI has only one resource element allocated for a pilot symbol.

[5]We wish to note that a fully convolutional architecture is not the optimal one for this simple task. The purpose of this experiment is merely to provide a simple setting for probing into the behavior of DeepRx, for which reason we did only minimal changes to its primary architecture.

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on July 08,2026 at 03:37:59 UTC from IEEE Xplore. Restrictions apply.

TABLE III

ABLATION STUDY RESULTS (UNCODED BER FOR UNIFORMLY DISTRIBUTED SINR BETWEEN 15 dB AND 20 dB). THE ARCHITECTURE WRITTEN IN BOLDFACE REFERS TO THE PRIMARY DEEPRX ARCHITECTURE (CF. TABLE I) USED TO GENERATE THE RESULTS IN SECTIONS V-A AND V-B. THE LABELS S, M, L AND XL IN THE ARCHITECTURE NAME REFER TO SMALL, MEDIUM, LARGE AND EXTRA LARGE IN TERMS OF NUMBER OF PARAMETERS

Name	Depth, ResNet Blocks	Parameters	Channels	Min-max dilation	BER (1 pilot, interference)	BER (2 pilots, interference)	BER (1 pilot, no interference)	BER (2 pilots, no interference)
Non-trainable baselines								
LMSSE					1.11×10^{-1}	3.38×10^{-3}	1.10×10^{-1}	2.60×10^{-3}
LMSSE with known channel					9.98×10^{-4}	9.62×10^{-4}	4.51×10^{-4}	4.81×10^{-4}
Large networks								
11 XL	11	7M	256-512	1-6	4.49×10^{-4}	3.77×10^{-4}		
11 L	11	3.4M	128-512	1-6	4.95×10^{-4}	4.14×10^{-4}		
13 L	13	1.8M	64-256	1-6	5.72×10^{-4}	4.71×10^{-4}		
Different depths								
13 M	13	1.2M	64-128	1-6	6.19×10^{-4}	4.93×10^{-4}	4.37×10^{-4}	4.14×10^{-4}
DeepRx	**11**	**1.2M**	**64-256**	**1-6**	$\mathbf{6.23 \times 10^{-4}}$	$\mathbf{4.98 \times 10^{-4}}$	$\mathbf{4.47 \times 10^{-4}}$	$\mathbf{4.18 \times 10^{-4}}$
5 M	5	1.2M	192-256	1-6	1.09×10^{-3}	7.36×10^{-4}	5.91×10^{-4}	5.13×10^{-4}
3 M	3	1.2M	256-448	1-6	1.63×10^{-3}	8.33×10^{-4}		
Depth multiplier (DM) 1 instead of 2 in all depthwise separable convolutions								
11 S-DM1	11	0.6M	64-128	1-6			4.63×10^{-4}	4.24×10^{-4}
Different widths								
11 S1	11	0.5M	64-128	1-6	7.72×10^{-4}	5.82×10^{-4}		
11 S2	11	0.3M	32-128	1-6	8.63×10^{-4}	6.32×10^{-4}		
11 S3	11	0.1M	16-64	1-6	3.72×10^{-1}	3.72×10^{-1}		
11 S4	11	0.06M	32	1-6	2.58×10^{-3}	1.23×10^{-3}		
No dilation (ND)								
11 M-ND	11	1.2M	64-256	1	6.91×10^{-4}	5.18×10^{-4}		
3 M-ND	3	1.2M	256-448	1	1.14×10^{-1}	1.17×10^{-3}		
Regular convolutions, i.e., not depthwise separable (C)								
11 M-C	11	2.7M	64-256	1-6	1.57×10^{-3}	8.71×10^{-4}		
Restricted CNN receiver, Fig. 10 (R)								
11 R	11	1.2M	64-256	1-6			7.00×10^{-2}	1.29×10^{-3}

model 11 XL with 7M parameters, and still observed some additional gains, but the performance increase started to level off after the 1M parameters mark. In order to explore smaller networks, we deviated from the general architecture a bit, and trained networks with constant width, and found a very small architecture (11 S4) with 32 convolutional channels, whose performance was relatively good (2.58×10^{-3}, 1 pilot). By default, all of the above models utilizing depthwise separable convolution [28], [29] use a depth multiplier value of 2, with the exception of one model (11 S-DM1), which uses a depth multiplier of 1 in order to demonstrate that DeepRx is robust to changing this hyperparameter.

In addition, as mentioned previously, we tested a network without depthwise separable convolutions (model 11 M-C) and a restricted network that has to perform the channel estimation without having access to the unknown data symbols (model 11 R, similar to Fig. 10). As shown in Table III, resorting to normal convolutions results in a slight drop in performance, while restricting the access to unknown data symbols deteriorated the performance to the level of the LMMSE baselines, as already observed in Section V-B.

As a final note, we emphasize that the computational complexity of DeepRx for scaling to larger TTIs is linear both in subcarrier and time dimensions (i.e., its asymptotic complexity can be expressed as $\mathcal{O}(SF)$), owing to the fully convolutional architecture. This applies to all the network architectures considered herein, as long as the pilot density within the TTI is kept roughly constant (otherwise receptive field might have to be adjusted to cover sparser pilots). On the other hand, the LMMSE receiver also has the complexity of $\mathcal{O}(SF)$ when making the reasonable assumption that its computational complexity is dominated by the equalization and demapping phase (see Section II). This means that the theoretical asymptotic complexity of DeepRx in terms of the reception bandwidth and TTI length is similar to that of the LMMSE receiver, although DeepRx has a larger constant multiplier not visible in the asymptotic expressions. Therefore, we expect that in practice DeepRx has a higher computational cost than the LMMSE receiver, although it also achieves superior radio performance. As for scaling DeepRx to support a larger amount of antennas, the impact on the computational complexity is more difficult to assess since it likely requires

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on July 08,2026 at 03:37:59 UTC from IEEE Xplore. Restrictions apply.

alterations to the network architecture such as increasing the number of convolutional filters for the first layers. Considering such multi-antenna aspects in more detail is left for future work.

VI. Conclusion

In this paper we considered a ML-based digital radio receiver, trained as a supervised training task from frequency domain antenna signals into uncoded bits, and implemented in a 5G-compliant manner. Our hypothesis and primary motivation behind the work was that training the nearly complete digital receiver chain as a single supervised system would result in higher performance compared to training multiple smaller parts of the receiver separately. This allows for optimizing the system directly for the end task of recovering the transmitted bits. In addition, restricting the neural network architecture as little as possible allows it to learn improved, and potentially unforeseen, receiver schemes. With this, it could learn to implicitly solve various radio channel and hardware impairments which might otherwise be challenging to capture.

To address and investigate the hypothesis, we implemented a deep fully convolutional neural network, referred to as DeepRx, which was trained to detect the received bits directly from the frequency-domain antenna signals. Moreover, DeepRx was trained to support different 5G-specific pilot configurations and modulation schemes. In contrast to many related works, the input of the neural network was constructed such that both the unknown data symbols as well as the known pilot symbols were arranged as convolutional input channels. This allowed DeepRx to efficiently combine both the data and pilot symbols when estimating the channel.

Through simulations modeling 5G uplink data transmission, we showed that the proposed DeepRx network outperforms traditional methods by a significant margin. It also outperformed an alternative neural network implementation where channel estimation and equalization were considered separately. We attributed the success primarily to DeepRx learning to utilize the known constellation points of the unknown data symbols, together with the local symbol distribution, to estimate and equalize the channel very accurately. In fact, some of the experiments indicated that the internal processing of DeepRx somewhat resembles that of iterative receivers. Moreover, it was also shown that DeepRx learns to deal efficiently with non-Gaussian interference and noise.

One restriction of this work is that it only includes a limited computational complexity analysis as the primary focus of this article is on the radio performance gains achieved by applying deep learning. Even though we have studied different approaches in improving the efficiency of the network, a further study on adapting these networks to inference time neural network chips is needed. Moreover, a fully comprehensive complexity comparison against conventional receivers is highly hardware-specific and needs to focus on latency and power consumption versus radio performance. While conducting such analysis is outside the scope of this article, it constitutes an important future work item for us. Another important future work topic is extending the DeepRx architecture to MIMO reception.

Acknowledgment

The authors would especially like to thank Vesa Starck for outstanding support and excellent insights during this work. The authors also thank Leo Kärkkäinen, Mikko Uusitalo, Jakob Hoydis, and Andrew Baldwin, for the various ideas and contributions.

References

[1] C. Zhang, P. Patras, and H. Haddadi, "Deep learning in mobile and wireless networking: A survey," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2224–2287, 3rd Quart., 2019.

[2] C. Jiang, H. Zhang, Y. Ren, Z. Han, K.-C. Chen, and L. Hanzo, "Machine learning paradigms for next-generation wireless networks," *IEEE Wireless Commun.*, vol. 24, no. 2, pp. 98–105, Apr. 2017.

[3] T. O'Shea and J. Hoydis, "An introduction to deep learning for the physical layer," *IEEE Trans. Cognit. Commun. Netw.*, vol. 3, no. 4, pp. 563–575, Dec. 2017.

[4] Y. Fu, S. Wang, C.-X. Wang, X. Hong, and S. McLaughlin, "Artificial intelligence to manage network traffic of 5G wireless networks," *IEEE Netw.*, vol. 32, no. 6, pp. 58–64, Nov. 2018.

[5] Z. Zhao, M. C. Vuran, F. Guo, and S. Scott, "Deep-waveform: A learned OFDM receiver based on deep complex convolutional networks," 2018, *arXiv:1810.07181*. [Online]. Available: https://arxiv.org/abs/1810.07181

[6] D. Neumann, T. Wiese, and W. Utschick, "Learning the MMSE channel estimator," *IEEE Trans. Signal Process.*, vol. 66, no. 11, pp. 2905–2917, Jun. 2018.

[7] M. E. Morocho-Cayamcela, H. Lee, and W. Lim, "Machine learning for 5G/B5G mobile and wireless communications: Potential, limitations, and future directions," *IEEE Access*, vol. 7, pp. 137184–137206, 2019.

[8] S. Chinchali *et al.*, "Cellular network traffic scheduling with deep reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 1–9.

[9] F. Tang *et al.*, "On removing routing protocol from future wireless networks: A real-time deep learning approach for intelligent traffic control," *IEEE Wireless Commun.*, vol. 25, no. 1, pp. 154–160, Feb. 2018.

[10] Z. Xu, Y. Wang, J. Tang, J. Wang, and M. C. Gursoy, "A deep reinforcement learning based framework for power-efficient resource allocation in cloud RANs," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2017, pp. 1–6.

[11] *Technical Specification Group Radio Access Network; NR; Physical Channels and Modulation*, docuemnt TS 38.211 Version 16.1.0 Release 16, 3GPP, ETSI, Sophia Antipolis Cedex, France, Mar. 2020.

[12] H. He, C.-K. Wen, S. Jin, and G. Y. Li, "Deep learning-based channel estimation for beamspace mmWave massive MIMO systems," *IEEE Wireless Commun. Lett.*, vol. 7, no. 5, pp. 852–855, Oct. 2018.

[13] Z. Chang, Y. Wang, H. Li, and Z. Wang, "Complex CNN-based equalization for communication signal," in *Proc. IEEE 4th Int. Conf. Signal Image Process. (ICSIP)*, Jul. 2019, pp. 513–517.

[14] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[15] Y. Bengio, "Learning deep architectures for AI," *Found. Trends Mach. Learn.*, vol. 2, no. 1, pp. 1–127, 2009.

[16] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, May 2015.

[17] O. Shental and J. Hoydis, "Machine LLRning: Learning to softly demodulate," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, Waikoloa, HI, USA, 2019.

[18] X. Gao, S. Jin, C.-K. Wen, and G. Y. Li, "ComNet: Combination of deep learning and expert knowledge in OFDM receivers," *IEEE Commun. Lett.*, vol. 22, no. 12, pp. 2627–2630, Dec. 2018.

[19] H. He, S. Jin, C.-K. Wen, F. Gao, G. Y. Li, and Z. Xu, "Model-driven deep learning for physical layer communications," *IEEE Wireless Commun.*, vol. 26, no. 5, pp. 77–83, Oct. 2019.

[20] N. Samuel, T. Diskin, and A. Wiesel, "Learning to detect," *IEEE Trans. Signal Process.*, vol. 67, no. 10, pp. 2554–2564, May 2019.

[21] H. Ye, G. Y. Li, and B.-H. Juang, "Power of deep learning for channel estimation and signal detection in OFDM systems," *IEEE Wireless Commun. Lett.*, vol. 7, no. 1, pp. 114–117, Feb. 2018.

[22] S. Dorner, S. Cammerer, J. Hoydis, and S. T. Brink, "Deep learning based communication over the air," *IEEE J. Sel. Topics Signal Process.*, vol. 12, no. 1, pp. 132–143, Feb. 2018.

[23] F. A. Aoudia and J. Hoydis, "End-to-end learning of communications systems without a channel model," in *Proc. 52nd Asilomar Conf. Signals, Syst., Comput.*, Pacific Grove, CA, USA, 2018, pp. 298–303.

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on July 08,2026 at 03:37:59 UTC from IEEE Xplore. Restrictions apply.

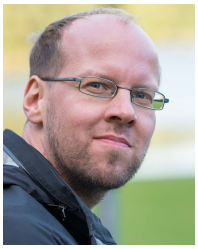
[24] S. Cammerer, F. A. Aoudia, S. Dörner, M. Stark, J. Hoydis, and S. ten Brink, "Trainable communication systems: Concepts and prototype," *IEEE Trans. Commun.*, vol. 68, no. 9, pp. 5489–5503, Sep. 2020.
[25] Mathworks. (2020). *MATLAB 5G Toolbox*. [Online]. Available: https://www.mathworks.com/products/5g.html
[26] *3rd Generation Partnership Project; Technical Specification Group Radio Access Network; Study on Channel Model for Frequencies From 0.5 to 100 GHz*, document TR 38.901 Version 16.0.0 Release 16, 3GPP, ETSI, Sophia Antipolis Cedex, France, Oct. 2019.
[27] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2016, pp. 630–645.
[28] L. Sifre and S. Mallat, "Rotation, scaling and deformation invariant scattering for texture discrimination," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.* Washington, DC, USA: IEEE Computer Society, Jun. 2013, pp. 1233–1240.
[29] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1800–1807.
[30] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, May 2016, pp. 1–14.
[31] Y. You *et al.*, "Large batch optimization for deep learning: Training BERT in 76 minutes," in *Proc. Int. Conf. Learn. Represent.*, 2020, pp. 1–38.
[32] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2017, *arXiv:1711.05101*. [Online]. Available: http://arxiv.org/abs/1711.05101
[33] Y. Léost, M. Abdi, R. Richter, and M. Jeschke, "Interference rejection combining in LTE networks," *Bell Labs Tech. J.*, vol. 17, no. 1, pp. 25–49, Jun. 2012.
[34] D. Godard, "Self-recovering equalization and carrier tracking in two-dimensional data communication systems," *IEEE Trans. Commun.*, vol. COM-28, no. 11, pp. 1867–1875, Nov. 1980.
[35] J. Bonnet and G. Auer, "Optimized iterative channel estimation for OFDM," in *Proc. IEEE Veh. Technol. Conf.*, Sep. 2006, pp. 1–5.



Mikko Honkala received the D.Sc. (Tech.) degree in computer science from the Helsinki University of Technology (HUT), Finland, in 2007. He has worked as a Machine Learning Researcher with the Nokia Research Center. He is currently with Nokia Bell Labs, where he leads the Machine Learning Methods and Applications Group. His research interest includes deep learning methods and their applications to new areas, such as wireless communications.



Dani Korpi received the D.Sc. (Tech.) degree (Hons.) in electrical engineering from the Tampere University of Technology (TUT), Finland, in 2017. He is currently a Senior Specialist with Nokia Bell Labs, Espoo, Finland. His research interests include machine learning for wireless communications, 6G radio systems, and in-band full-duplex radios. He received the Best Dissertation Award of the Year from TUT and the Finnish Technical Sector's Award for the Best Doctoral Dissertation of 2017.



Janne M. J. Huttunen received the M.Sc. degree in medical physics and the Ph.D. degree in physics from the University of Kuopio (UKU), Finland, in 2004 and 2008, respectively. Previously, he has worked as an Academic Researcher developing methods for Bayesian inverse problems and applications. He is currently a Senior Researcher with Nokia Bell Labs, Espoo, Finland. His research interest includes deep learning in various areas, including wireless communications. He received the Best Dissertation Award of the Year from UKU.

Authorized licensed use limited to: CAMBRIDGE UNIV. Downloaded on July 08,2026 at 03:37:59 UTC from IEEE Xplore. Restrictions apply.